



Mastering kubectl Workflows for Speed and Precision



Tim Warner

Principal Author, Pluralsight

TechTrainerTim.com



Four framing questions



How do imperative commands let me create resources in seconds instead of writing YAML?



How does the dry-run-to-YAML pipeline bridge speed and flexibility?



How do kubectl explain and resource querying save minutes during troubleshooting?



Why is context switching the number one CKA exam gotcha?





“I need 12 resources deployed before the stand-up, and you have got 15 minutes. Writing YAML from scratch is not going to cut it. Oh, and remember -- we run six clusters now. Deploy to the wrong one and nobody will catch it until production breaks.”



**How do imperative commands let me
create resources in seconds?**



Imperative commands: the exam speed toolkit

kubectl can create a pod quickly (one-off/debug)

kubectl create deployment sets replica count without YAML

kubectl expose creates ClusterIP, NodePort, or LoadBalancer Services

kubectl create configmap and secret inject configuration data

kubectl create role + rolebinding set up RBAC access control

These are high-frequency patterns used repeatedly on exam

kubectl scale deployment web --replicas=5



kubectl talks to the API server for cluster operations; kubectl config works on kubeconfig locally.

Exam tip



When imperative commands are not enough

Resource limits, nodeSelector, tolerations → typically YAML

Multi-container Pods require YAML (multiple container specs)

Kubernetes supports imperative creation for some objects, but not all

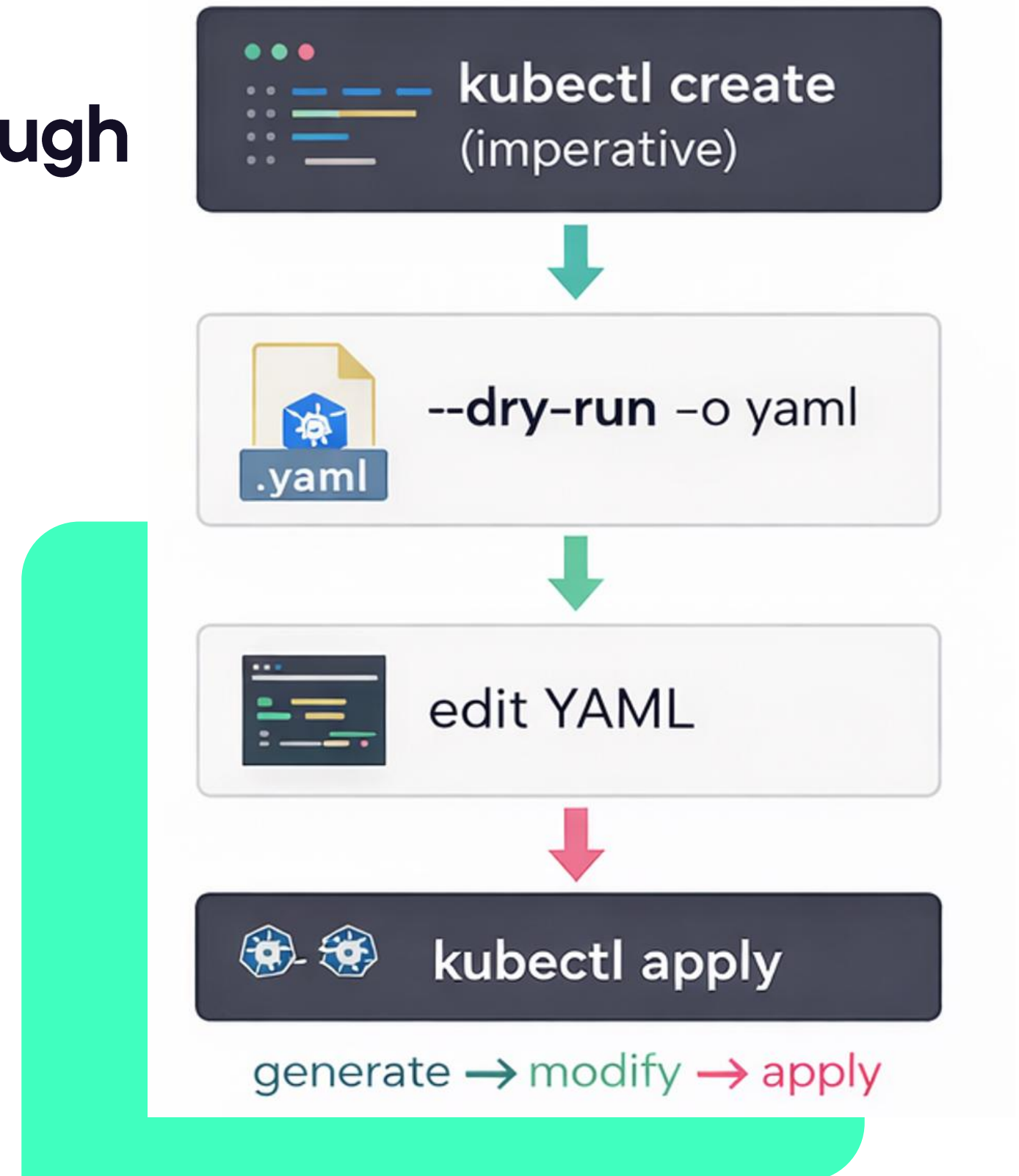
NetworkPolicy, PersistentVolume, DaemonSet → YAML only

PSI browser limited to allowed docs:

Kubernetes docs/blog

Helm docs

Gateway API



Use dry-run + explain to generate and complete these manifests



Use imperative first – fall back to
YAML only when required.

Exam tip



**How does the dry-run-to-YAML pipeline
bridge speed and flexibility?**



You don't write YAML from scratch – you generate it and modify it.

Exam tip



The dry-run-to-YAML pipeline

Generate
YAML from
command

No API call
(safe
preview)

Full
manifest
output

Edit
required
fields

Apply to
create
resources

```
kubectl run temp --image=busybox --restart=Never --dry-run=client -o yaml > pod.yaml
```

--dry-run=client builds the resource locally without creating it in the cluster

-o yaml prints a complete, valid Kubernetes manifest structure

Add missing fields like resources, labels, nodeSelector, or tolerations

```
kubectl apply -f pod.yaml creates the resource from your edited manifest
```



This is your fastest path from idea
→ valid YAML → applied
resource.

Exam tip



kubectl explain: your offline API reference

`kubectl explain
pod.spec.contain
ers` shows field-level
documentation

`kubectl explain
--recursive` prints
the full field tree for
nested discovery

`kubectl explain
pod.spec.contain
ers.resources` for
resource limit field
paths

`kubectl api-resources`



kubectl explain: your offline API reference

kubectl explain
deployment.spec.strategy –
recursive for rollout options

Faster than searching docs for
most field lookups on the CKA
exam



You don't always want to leave the terminal – explain keeps you in flow.

Exam tip



How do I query resources efficiently?



Resource querying: get exactly what you need

Label selectors

- `kubectl get pods -l app=web,env=prod`

Field selectors

- `kubectl get pods -field-selector status.phase=Running`

Minimal output

- `kubectl get pods -o name`

Custom columns

- `kubectl get pods -o custom-columns=NAME:metadata.name,NODE:.spec.nodeName`

Chronological debugging

- `kubectl get pods --sort-by=.metadata.creationTimestamp`



Default output is for humans –
selectors and JSONPath are for
speed.

Exam tip



**Why is context switching the number
one CKA exam gotcha?**



Context management: the silent exam killer

The CKA exam uses 15-20 performance-based tasks over 2 hours

Each task specifies a context – you must switch before working

`kubectl config get-contexts` shows all clusters (asterisk marks active)



Context management: the silent exam killer

`kubectl config use-context` switches to the correct cluster

`kubectl config set-context --current --namespace=production`
sets the default namespace

`kubectl config current-context` shows the active cluster context



Demo

**kubectl speed run: imperative
creation, dry-run pipeline,
querying, and context switching**





“Remember those 12 resources I needed before stand-up? You just did it in under a minute. Imperative commands, context switching on autopilot -- that is exactly the muscle memory that separates a confident admin from someone who panics under the clock.”





From Globomantics to you



Master the imperative toolkit

Six commands cover 60% of exam tasks -- use them for everything they support



Build the dry-run reflex

Generate, edit, apply -- this pipeline and kubectl explain handle everything else



Query with precision

Label selectors, JSONPath, and custom-columns save minutes over scrolling default output



Switch context on autopilot

Wrong cluster means zero points -- make it muscle memory, not a checkbox

